Universidad Católica de Temuco

INFORME DE PROYECTO

Diseño e implementación de un analizador sintáctico

Teoría de la Computación

Código curso	INFO1148
Integrantes	Nataly Huaiquinao Sáez Diego Sanhueza Reveco Alvaro Valenzuela Paillacar
Profesor	Prof. M. Lévano
Fecha de entrega	01 de Noviembre, 2026

Temuco, 2026

Resumen

Este proyecto presenta el diseño y la implementación en Python de un analizador sintáctico para un lenguaje de dominio específico de automatización agrícola. El lenguaje permite definir reglas que relacionan condiciones sobre sensores con acciones de activación, desactivación y riego temporizado. Su especificación incluye operadores relacionales y lógicos, agrupación con paréntesis y múltiples acciones por regla. Se construyó una gramática libre de contexto con 12 no terminales y 22 producciones; se analizó su recursión, factorización y precedencia, y se calcularon FIRST y FOLLOW para elaborar una tabla predictiva LL(1) con 29 celdas ocupadas y sin conflictos. La implementación combina un lexer sencillo con un parser manual de descenso recursivo que construye un AST y comunica errores localizados. La verificación desde una copia limpia registró 44 pruebas unitarias e integración correctas y un corpus de 25 entradas con resultados esperados: 10 válidas, 10 inválidas y 5 casos límite. La solución reconoce estructura, pero no verifica tipos, existencia de sensores ni seguridad agronómica. Se entregan código, ejemplos e instrucciones de reproducción en GitHub.

1. Introducción y dominio seleccionado

El análisis sintáctico verifica que una secuencia de tokens cumpla la estructura definida por una gramática libre de contexto (GLC) y, cuando es válida, construye una representación jerárquica de la entrada. Es la etapa que sigue al análisis léxico y la base para interpretar o compilar un lenguaje.

En este proyecto diseñamos un mini lenguaje de dominio específico para la automatización agrícola, y desarrollamos su analizador sintáctico en Python, sin utilizar generadores automáticos de parsers.

1.1 Dominio escogido

El dominio escogido es la automatización agrícola, específicamente la descripción de reglas de riego y control de dispositivos a partir de variables como humedad del suelo, temperatura y presencia de lluvia. En este contexto, se necesita expresar de forma clara qué condición debe cumplirse y qué acción corresponde realizar.

Elegimos este dominio porque permite relacionar conceptos de teoría de la computación con situaciones concretas de un cultivo. El lenguaje está pensado como un prototipo académico para representar reglas que un encargado de riego podría revisar y configurar, sin tener que escribir un programa de propósito general.

1.2 Propósito del lenguaje

El lenguaje permite escribir una o más reglas con la estructura WHEN condición THEN acciones;. Las condiciones comparan sensores con enteros no negativos o valores booleanos y pueden combinarse mediante AND, OR, NOT y paréntesis. Las acciones permiten activar o desactivar un dispositivo y solicitar el riego de un sector durante una cantidad entera de minutos; la palabra Y une varias acciones en una misma regla. El alcance de esta versión es reconocer la estructura de las reglas y construir su árbol de sintaxis abstracta (AST). No ejecuta acciones sobre equipos reales ni comprueba que los sensores existan, que sus tipos sean compatibles o que los umbrales y tiempos sean adecuados para un cultivo.

1.3 Objetivos

Objetivo general. Diseñar e implementar en Python un analizador sintáctico para un mini lenguaje de automatización agrícola, aplicando gramáticas libres de contexto, conjuntos FIRST y FOLLOW, análisis LL(1), construcción de AST y detección de errores sintácticos.

Objetivos específicos.

- Definir formalmente el lenguaje: tokens, palabras reservadas, operadores y ejemplos válidos e inválidos.
- Construir una GLC coherente con el lenguaje y analizarla (recursión izquierda, factorización, ambigüedad).
- Calcular FIRST y FOLLOW y construir la tabla predictiva LL(1).
- Implementar un parser predictivo con construcción de AST y mensajes de error útiles.
- Validar la solución con un conjunto de pruebas válidas, inválidas y de casos límite.

1.4 Organización del informe

El informe presenta la especificación del lenguaje, su gramática y el análisis de ella, los conjuntos FIRST y FOLLOW, la tabla LL(1), el diseño del parser, el árbol sintáctico, el manejo de errores, las pruebas y las conclusiones. Los anexos incluyen el repositorio, las instrucciones de ejecución y las capturas de la demostración.

2. Especificación del lenguaje

2.1 Alfabeto y categorías de tokens

El alfabeto relevante comprende letras ASCII (A–Z y a–z), dígitos (0–9), guion bajo y los símbolos >, <, =, !, (,) y ;. Los símbolos = y ! solo se admiten dentro de comparadores como == o !=. Los espacios, tabulaciones y saltos de línea separan tokens y se omiten al analizar la estructura. No se admiten tildes en identificadores, cadenas de texto, comentarios, números negativos ni decimales. Las categorías léxicas son las siguientes:

Token	Patrón / lexema	Ejemplos	Observaciones
Reservadas	Lexemas de la sección 2.2	WHEN, REGAR, TRUE	Cada reservada genera su propio tipo de token; no distingue mayúsculas.
ID	[A-Za-z_][A-Za-z0-9_]*	humedad_suelo, sector_1	Excluye reservadas y conserva la escritura original.
NUM	[0-9]+	0, 15, 30	Enteros no negativos; sin signo ni parte decimal.
REL	>= | <= | == | != | > | <	<, >=, ==	Se reconoce el comparador completo y se conserva su lexema.
Puntuación	( ) ;	(humedad < 30);	Los paréntesis agrupan condiciones y ; termina una regla.

Token	Patrón / lexema	Ejemplos	Observaciones
Fin de entrada	$	Token interno $	Lo agrega el lexer; no se escribe en el programa fuente.

2.2 Palabras reservadas, operadores y separadores

Elemento	Lista
Palabras reservadas	WHEN, THEN, AND, OR, NOT, TRUE, FALSE, ACTIVAR, DESACTIVAR, REGAR, POR, MIN, Y.
Identificadores	Una letra ASCII o guion bajo inicial, seguida de letras ASCII, dígitos o guiones bajos. Ejemplos: bomba, lluvia, sector_norte. No pueden coincidir con una reservada.
Valores	Enteros no negativos y booleanos TRUE o FALSE. La duración del riego utiliza exclusivamente un entero.
Operadores	Relacionales: >, <, >=, <=, ==, !=. Lógicos: NOT, AND y OR. Precedencia: agrupación, NOT, AND y OR; AND y OR se asocian a la izquierda en el AST.
Separadores	; termina cada regla; (y) agrupan condiciones. Los espacios en blanco separan tokens. Y une acciones y AND une condiciones.

2.3 Requisitos mínimos del lenguaje

La siguiente tabla relaciona los requisitos mínimos del enunciado con las construcciones del lenguaje. Un programa contiene al menos una regla, y cada regla incluye al menos una acción.

Requisito	Cómo se cumple
Al menos 4 palabras reservadas	Se definen 13: WHEN, THEN, AND, OR, NOT, TRUE, FALSE, ACTIVAR, DESACTIVAR, REGAR, POR, MIN y Y.
Identificadores	Los sensores, dispositivos y sectores se nombran mediante ID; por ejemplo, humedad_suelo, bomba y norte.
Valores numéricos y/o booleanos	Se admiten enteros no negativos, como 30 y 15, y los booleanos TRUE y FALSE.
Al menos 3 operadores	Se incluyen seis comparadores relacionales y tres operadores lógicos: AND, OR y NOT.
Estructura de sentencia principal	WHEN condición THEN acciones;. Ejemplo: WHEN humedad < 30 THEN ACTIVAR bomba;
Más de una sentencia o regla por programa	Se pueden concatenar reglas completas, cada una terminada en ;. No se admite un programa vacío.
Al menos una construcción compuesta	Se combinan condiciones con AND y OR, se agrupan con paréntesis y se unen acciones con Y. REGAR incluye el parámetro de duración POR NUM MIN.

2.4 Ejemplos de sentencias

Sentencias válidas (mínimo 5).

N.º	Sentencia	Por qué es válida
1	WHEN humedad < 30 THEN ACTIVAR bomba;	Comparación numérica y una acción con destino.
2	WHEN humedad >= 70 THEN DESACTIVAR bomba;	Comparador >= y desactivación de un dispositivo.
3	WHEN lluvia == FALSE THEN REGAR norte POR 15 MIN;	Comparación booleana y riego con duración completa.
4	WHEN humedad < 30 AND temperatura > 20 THEN ACTIVAR bomba;	Dos comparaciones unidas por AND.
5	WHEN NOT (lluvia == TRUE OR humedad >= 60) THEN REGAR norte POR 10 MIN Y ACTIVAR bomba;	Negación de una condición agrupada y dos acciones unidas por Y.

Sentencias inválidas (mínimo 5).

N.º	Sentencia	Por qué es inválida
1	WHEN humedad < 30 ACTIVAR bomba;	Falta THEN entre la condición y la acción.
2	WHEN humedad < THEN ACTIVAR bomba;	Falta un valor después del comparador <.
3	WHEN humedad < 30 THEN ACTIVAR bomba	Falta el punto y coma al final de la regla.
4	WHEN (humedad < 30 THEN ACTIVAR bomba;	Falta cerrar el paréntesis antes de THEN.
5	WHEN humedad < 30 THEN REGAR norte 10 MIN;	Falta POR antes de la duración del riego.

2.5 Componente léxico

El componente léxico adapta el analizador básico entregado por el docente en anlizador_lexico1.ipynb. En src/agro.py, tokenizar utiliza expresiones regulares para reconocer números, comparadores, identificadores, puntuación y espacios. Los identificadores se contrastan con las reservadas usando su versión en mayúsculas. Cada Token guarda tipo, lexema, línea y columna, con posiciones desde 1. Los comparadores de dos caracteres se reconocen antes que los de uno, los espacios se omiten y al final se agrega $. Un símbolo no reconocido produce ErrorLexico localizado. La lista resultante se entrega al parser; los tipos compartidos y las excepciones están definidos en src/contratos.py.

3. Gramática libre de contexto

La gramática libre de contexto se define como G = (N, T, P, Programa), donde N es el conjunto de no terminales, T el de terminales y P las producciones de la sección 3.2. Esta versión corresponde a la gramática candidata publicada en src/gramatica.py y al lenguaje descrito en la sección 2. Las producciones se expresan sobre tipos de tokens; por ejemplo, ID representa un identificador y REL cualquiera de los seis comparadores.

3.1 Elementos de la gramática

Elemento	Contenido
Símbolo inicial	Programa
No terminales	N = {Programa, Resto, Regla, Expr, ColaOr, Termino, ColaAnd, Factor, Comparacion, Valor, Accion, MasAcciones}.
Terminales	T = {WHEN, THEN, OR, AND, NOT, (,), ID, REL, NUM, TRUE, FALSE, ACTIVAR, DESACTIVAR, REGAR, POR, MIN, Y, ;}.

3.2 Producciones

Programa → Regla Resto
Resto → Regla Resto | ε
Regla → WHEN Expr THEN Accion MasAcciones ;
Expr → Termino ColaOr
ColaOr → OR Termino ColaOr | ε
Termino → Factor ColaAnd
ColaAnd → AND Factor ColaAnd | ε
Factor → NOT Factor | (Expr) | Comparacion
Comparacion → ID REL Valor
Valor → NUM | TRUE | FALSE
Accion → ACTIVAR ID | DESACTIVAR ID | REGAR ID POR NUM MIN
MasAcciones → Y Accion MasAcciones | ε

3.3 Gramática original y gramática transformada

Para justificar el diseño, se comparan formas sencillas de expresar reglas y condiciones con sus versiones transformadas. Son ejemplos de construcción de la gramática, no un registro de versiones anteriores del grupo.

La lista de reglas podría escribirse como Programa → Regla Programa | Regla. Ambas alternativas comparten el prefijo Regla. Se factoriza en Programa → Regla Resto y Resto → Regla Resto | ε: se conserva una o más reglas, y la decisión de continuar se toma después de reconocer la primera.

Para expresar OR, la forma Expr → Expr OR Termino | Termino tiene recursión izquierda directa. Su transformación es Expr → Termino ColaOr y ColaOr → OR Termino ColaOr | ε. Para AND se aplica el mismo procedimiento: Termino → Factor ColaAnd y ColaAnd → AND Factor ColaAnd | ε, en lugar de Termino → Termino AND Factor | Factor.

La transformación conserva las secuencias reconocidas en cada nivel. Aunque la gramática usa colas recursivas por la derecha, los métodos cola_or y cola_and del parser consumen esas repeticiones mediante ciclos y pliegan el AST a la izquierda, de acuerdo con la asociación definida en la sección 2.

4. Análisis de la gramática

4.1 Terminales, no terminales y símbolo inicial

Se utilizan exactamente los conjuntos N y T de la sección 3.1 y Programa como símbolo inicial: 12 no terminales, 19 terminales y 22 producciones, contando por separado cada alternativa. ε denota la cadena vacía y no es un terminal. En el código se representa mediante la tupla vacía (). El marcador $ indica fin de entrada para el análisis predictivo, pero tampoco pertenece a T ni se escribe en el programa fuente.

Resto, ColaOr, ColaAnd y MasAcciones pueden derivar ε. Programa no es anulable: exige una Regla; esta consume WHEN, una condición, THEN, al menos una acción y el terminador ;. Todos los no terminales son alcanzables desde Programa y pueden derivar alguna secuencia terminal.

4.2 Recursión izquierda

Existe recursión izquierda si un no terminal A puede derivar Aα sin consumir antes un terminal. En las formas didácticas de la sección 3.3, Expr y Termino presentan recursión izquierda directa; las colas ColaOr y ColaAnd permiten eliminarla.

La gramática final no tiene recursión izquierda directa. Las repeticiones en Resto, ColaOr, ColaAnd y MasAcciones consumen, respectivamente, una regla que comienza en WHEN, o los tokens OR, AND y Y antes de repetirse. Factor → NOT Factor también consume NOT antes de la llamada recursiva, y Factor → (Expr) consume el paréntesis de apertura.

Para comprobar la ausencia de recursión izquierda indirecta, se revisan los no terminales que pueden aparecer al inicio de una derivación, saltando prefijos anulables. Las cadenas principales son Programa → Regla, Resto → Regla y Expr → Termino → Factor → Comparacion. Ninguna vuelve al no terminal de partida sin consumir un terminal. Los no terminales anulables de esta gramática aparecen en las colas de las producciones y no introducen un ciclo de este tipo.

4.3 Factorización por la izquierda

La factorización por la izquierda separa alternativas con un prefijo común. En Programa → Regla Programa | Regla, la forma factorizada extrae Regla y deja en Resto la elección entre continuar con otra regla o terminar. Este cambio se refleja en la gramática de la sección 3.2.

En la versión final no quedan alternativas no vacías con prefijos iniciales comunes que requieran nueva factorización. Factor distingue NOT, (e ID; Valor distingue NUM, TRUE y FALSE; Accion distingue ACTIVAR, DESACTIVAR y REGAR. Las colas distinguen una continuación por WHEN, OR, AND o Y frente a ε. La selección de sus alternativas vacías se justificará con FOLLOW y con la tabla predictiva en las secciones 5 y 6.

4.4 Ambigüedad

Una gramática es ambigua si una misma secuencia terminal admite dos árboles de derivación distintos. Una regla plana como Cond → Cond AND Cond | Cond OR Cond | Comparación no fija prioridades: la condición humedad < 30 OR temperatura > 32 AND lluvia == FALSE podría interpretarse agrupando primero OR o primero AND.

La gramática propuesta separa los niveles Expr, Término y Factor. OR conecta términos; AND conecta factores; NOT se aplica a un factor, que puede ser otra negación, una comparación o un grupo entre paréntesis. Por ello, el ejemplo se agrupa como (humedad < 30) OR ((temperatura > 32) AND (lluvia == FALSE)). Para cambiar esta agrupación se deben escribir los paréntesis correspondientes.

NOT humedad < 30 se interpreta como NOT (humedad < 30), porque Comparación reconoce ID REL Valor como una unidad. Los booleanos aislados, como WHEN TRUE THEN ..., no forman una condición válida. En cadenas con el mismo operador, la gramática fija una única secuencia de cola; el AST debe asociar AND y OR a la izquierda. Esta asociación del AST no debe confundirse con la recursión por la derecha de las producciones.

Las acciones también tienen un inicio distinguible y Y las separa; no se confunden con las condiciones unidas por AND. Estas decisiones fijan la estructura prevista. La comprobación formal de que las alternativas pueden seleccionarse sin conflictos se presentará al construir la tabla LL(1), en la sección 6.

5. Conjuntos FIRST y FOLLOW

5.1 Conjuntos FIRST

FIRST(A) reúne los terminales que pueden iniciar una cadena derivada de A; incluye ε si A puede derivar la cadena vacía. Para una secuencia se examinan los símbolos de izquierda a derecha y se continúa mientras sean anulables. El cálculo se repite hasta que los conjuntos dejan de cambiar.

No terminal	FIRST	Justificación
Programa	{WHEN}	Regla inicia con WHEN y no es anulable.
Resto	{WHEN, ε}	Regla inicia con WHEN; la otra alternativa es ε.
Regla	{WHEN}	La producción comienza con WHEN.
Expr	{(, ID, NOT}	Termino no es anulable; se hereda su FIRST.
ColaOr	{OR, ε}	Sus alternativas comienzan con OR o son ε.
Termino	{(, ID, NOT}	Factor no es anulable; se hereda su FIRST.
ColaAnd	{AND, ε}	Sus alternativas comienzan con AND o son ε.

No terminal	FIRST	Justificación
Factor	{(, ID, NOT}	NOT y (son inicios directos; Comparacion aporta ID.
Comparacion	{ID}	La comparación comienza con ID.
Valor	{FALSE, NUM, TRUE}	Cada alternativa comienza con un valor diferente.
Accion	{ACTIVAR, DESACTIVAR, REGAR}	Cada acción comienza con una reservada distinta.
MasAcciones	{Y, ε}	La continuación comienza con Y o desaparece mediante ε.

5.2 Conjuntos FOLLOW

FOLLOW(A) reúne los terminales que pueden aparecer inmediatamente después de A en una derivación; incluye $ cuando A puede quedar al final. Para B → α A β se incorpora FIRST(β) sin ε y, si β es anulable, FOLLOW(B). Se inicia FOLLOW(Programa) con $ y se propaga hasta alcanzar un punto fijo. ε nunca pertenece a FOLLOW.

No terminal	FOLLOW	Justificación
Programa	{$}	Símbolo inicial; puede terminar la entrada.
Resto	{$}	Está al final de Programa; hereda $.
Regla	{$, WHEN}	Resto aporta WHEN y, al ser anulable, permite heredar $.
Expr	{), THEN}	En Regla le sigue THEN; dentro de Factor le sigue).
ColaOr	{), THEN}	Está al final de Expr; hereda su FOLLOW.
Termino	{), OR, THEN}	ColaOr aporta OR y, al ser anulable, FOLLOW(Expr).
ColaAnd	{), OR, THEN}	Está al final de Termino; hereda su FOLLOW.
Factor	{), AND, OR, THEN}	ColaAnd aporta AND y, al ser anulable, FOLLOW(Termino).
Comparacion	{), AND, OR, THEN}	Está al final de Factor; hereda su FOLLOW.
Valor	{), AND, OR, THEN}	Está al final de Comparacion; hereda su FOLLOW.
Accion	{;, Y}	MasAcciones aporta Y y, al ser anulable, permite heredar ;.
MasAcciones	{;}	En Regla le sigue el terminador ;.

6. Tabla LL(1)

6.1 Construcción

Para cada producción $A \rightarrow \alpha$ se incorpora esa producción en $M[A, a]$ para cada $a \in FIRST(\alpha) - \{\varepsilon\}$. Si α deriva ε, se incorpora además en $M[A, b]$ para cada $b \in FOLLOW(A)$. Una celda con dos producciones distintas es un conflicto; una celda sin producción indica error sintáctico.

6.2 Tabla predictiva

La tabla se presenta en formato disperso: cada fila identifica una celda ocupada mediante el no terminal, el token de anticipación y el número de producción. Las celdas no listadas son error. Los números remiten al catálogo que aparece a continuación; $ es fin de entrada y REL agrupa los seis comparadores.

No terminal	Anticipación	N.º de producción
Programa	WHEN	1
Resto	WHEN	2
Resto	$	3
Regla	WHEN	4
Expr	(	5
Expr	ID	5
Expr	NOT	5
ColaOr	OR	6
ColaOr	)	7
ColaOr	THEN	7
Termino	(	8
Termino	ID	8
Termino	NOT	8
ColaAnd	AND	9
ColaAnd	)	10
ColaAnd	OR	10
ColaAnd	THEN	10
Factor	NOT	11
Factor	(	12
Factor	ID	13
Comparacion	ID	14
Valor	NUM	15
Valor	TRUE	16
Valor	FALSE	17
Accion	ACTIVAR	18
Accion	DESACTIVAR	19
Accion	REGAR	20
MasAcciones	Y	21
MasAcciones	;	22

N.º	Producción	N.º	Producción
1	Programa → Regla Resto	12	Factor → (Expr)
2	Resto → Regla Resto	13	Factor → Comparacion
3	Resto → ε	14	Comparacion → ID REL Valor
4	Regla → WHEN Expr THEN Accion MasAcciones ;	15	Valor → NUM
5	Expr → Termino ColaOr	16	Valor → TRUE
6	ColaOr → OR Termino ColaOr	17	Valor → FALSE
7	ColaOr → ε	18	Accion → ACTIVAR ID
8	Termino → Factor ColaAnd	19	Accion → DESACTIVAR ID
9	ColaAnd→ AND Factor ColaAnd	20	Accion → REGAR ID POR NUM MIN
10	ColaAnd→ ε	21	MasAcciones → Y Accion MasAcciones
11	Factor → NOT Factor	22	MasAcciones → ε

6.3 ¿La gramática es LL(1)?

Sí, la gramática es LL(1): cada celda de su tabla predictiva contiene a lo sumo una producción. El cálculo produce 29 celdas ocupadas sin conflictos. El código puede conservar varias candidatas para detectar conflictos, pero en esta gramática todas las celdas ocupadas tienen una sola.

Las 18 producciones no vacías ocupan 22 celdas y las cuatro producciones ε ocupan siete celdas, para un total de 29. La ausencia de conflictos se justifica por la separación de inicios y por las condiciones de las alternativas anulables que se detallan a continuación.

En Factor, las alternativas comienzan con NOT, (e ID; en Valor, con NUM, TRUE y FALSE; y en Accion, con ACTIVAR, DESACTIVAR y REGAR. Los FIRST de las alternativas de cada no terminal son disjuntos.

Para los no terminales con una alternativa vacía, el FIRST de la alternativa no vacía es disjunto de su FOLLOW: Resto usa {WHEN} frente a {$}; ColaOr, {OR} frente a {THEN,)}; ColaAnd, {AND} frente a {OR, THEN,)}; y MasAcciones, {Y} frente a {;}. Por ello, continuar o seleccionar ε no introduce dos producciones en una misma celda.

Programa, Regla, Expr, Termino y Comparacion tienen una sola producción, por lo que no presentan competencia entre alternativas.

El analizador puede seleccionar una producción con un solo token de anticipación. Este resultado completa el análisis de la sección 4: la gramática fija prioridades, carece de recursión izquierda y su tabla predictiva confirma que no hay conflictos LL(1).

7. Diseño e implementación del parser

7.1 Estrategia

El analizador es predictivo y de descenso recursivo: cada no terminal de la gramática se implementa como una función que decide qué producción aplicar según el token actual (lookahead), de acuerdo con la tabla LL(1). No se utilizó ninguna biblioteca que genere el parser automáticamente.

7.2 Organización del código

Módulo / archivo	Responsabilidad
src/agro.py y src/contratos.py	Tokenizar el texto; compartir Token, constantes y errores localizados.
src/parser.py	Consumir tokens mediante descenso recursivo y construir el AST respetando la gramática.
src/arbol.py	Representar nodos con etiqueta e hijos y dibujar el AST mediante una pila explícita.
src/main.py y src/verificar_entrega.py	Ejecutar archivos por CLI; analizar el corpus y guardar resultados reproducibles.
src/gramatica.py, src/analisis_gramatica.py, src/exportar_analisis.py y tests/	Definir producciones, calcular conjuntos y tabla, exportar el análisis y verificar módulos e integración.

7.3 Correspondencia entre gramática y código

La correspondencia entre gramática y código es explícita. En Parser.regla(), se consume WHEN, se llama a expr(), se exige THEN, se reconoce accion() y se acumulan mas_acciones() antes de exigir ;. Parser.factor() elige según el token NOT, (o ID; en los dos primeros casos reconoce la negación o un grupo y, en el tercero, llama a comparacion(). Las colas AND/OR se implementan con ciclos equivalentes a las producciones recursivas, conservando la asociación a la izquierda.

Producción	Función en el código
Regla → WHEN Expr THEN Accion MasAcciones ;	Parser.regla(): consumir('WHEN'), expr(), consumir('THEN'), accion(), mas_acciones(), consumir(';').
Factor → NOT Factor | (Expr) | Comparacion	Parser.factor(): distingue NOT, (e ID; construye NOT, devuelve el grupo o reconoce comparacion().

7.4 Instrucciones de ejecución

Se requiere Python 3.10 o superior y no hay dependencias externas de ejecución. Desde la raíz del repositorio se utiliza python -m src.main ejemplos/validos/01_activar.txt; --tokens permite mostrar el análisis léxico. Los archivos se leen como UTF-8 con o sin BOM. La salida incluye el AST si el programa es válido; ante errores de lectura, léxicos o sintácticos, el CLI devuelve código 1. Una ejecución correcta devuelve 0. README.md contiene los comandos completos.

8. Árbol sintáctico / AST

El parser construye un AST con objetos Nodo, cada uno con una etiqueta y una lista ordenada de hijos. El siguiente árbol corresponde a ejemplos/validos/05_valido.txt y se obtuvo de una ejecución real; también está guardado en resultados/ast_ejemplo.txt.

WHEN NOT (lluvia == TRUE OR humedad >= 60) THEN REGAR norte POR 10 MIN Y ACTIVAR bomba;

```
Programa
└── Regla
    ├── NOT
    │   └── OR
    │       ├── ==
    │       │   ├── lluvia
    │       │   └── TRUE
    │       └── >=
    │           ├── humedad
    │           └── 60
    └── Acciones
        ├── REGAR norte (10 MIN)
        └── ACTIVAR bomba
```

programa() reúne las reglas; regla() produce una condición y un nodo Acciones; las funciones de expresión crean nodos OR, AND y NOT; comparacion() conserva el comparador, sensor y valor; y accion() conserva la operación, destino y duración cuando existe. Es un AST y no un árbol de derivación completo: omite WHEN, THEN, ; y los paréntesis, pero conserva su efecto sobre la estructura. La representación textual recorre los nodos sin recursión.

9. Manejo de errores sintácticos

El analizador no se limita a responder «error»: informa qué se esperaba, qué se encontró y en qué posición.

Entrada inválida	Mensaje emitido	Posición
WHEN humedad < 30 ACTIVAR bomba;	se esperaba THEN y se encontró 'ACTIVAR'	1:19
WHEN humedad < THEN ACTIVAR bomba;	se esperaba un valor (NUM, TRUE o FALSE) después del operador < y se encontró 'THEN'	1:16
WHEN humedad < 30 THEN ACTIVAR bomba; WHEN a @ 3 THEN ACTIVAR b;	símbolo no reconocido: '@'	2:8

Se detiene el análisis en el primer error. El lexer conserva línea y columna desde 1 y actualiza la posición al encontrar saltos de línea; los archivos se leen normalizando sus saltos de línea. ErrorLexico y ErrorSintactico comparten mensaje y ubicación. El parser informa el token esperado y el encontrado, incluido fin de entrada cuando falta un símbolo final. La API también rechaza listas de tokens sin un único marcador $ al final. Ante una profundidad excesiva de paréntesis o NOT, el CLI informa el límite de Python sin mostrar una traza interna.

10. Casos de prueba y extensión creativa

10.1 Extensión creativa

La extensión principal combina riego temporizado y múltiples acciones: REGAR ID POR NUM MIN incorpora una duración obligatoria, y MasAcciones → Y Accion MasAcciones | ε permite agregar operaciones a una misma regla. Estas construcciones aparecen en la gramática, FIRST(Accion), FIRST/FOLLOW(MasAcciones), la tabla LL(1), Parser.accion(), Parser.mas_acciones() y los nodos del AST. Los ejemplos V05 y V10 combinan acciones, mientras I05, I06 e I10 verifican errores de duración o continuación. También se incluyen NOT y agrupación con paréntesis, comprobados por pruebas de estructura y precedencia.

10.2 Casos válidos (mínimo 10)

ID	Entrada	Esperado	Obtenido	Observación
V01	WHEN humedad < 30 THEN ACTIVAR bomba;	Válida	Válida	Comparación y activación.
V02	WHEN humedad >= 70 THEN DESACTIVAR bomba;	Válida	Válida	Comparador >= y desactivación.
V03	WHEN lluvia == FALSE THEN REGAR norte POR 15 MIN;	Válida	Válida	Booleano y duración.
V04	WHEN humedad < 30 AND temperatura > 20 THEN ACTIVAR bomba;	Válida	Válida	Condición con AND.
V05	WHEN NOT (lluvia == TRUE OR humedad >= 60) THEN REGAR norte POR 10 MIN Y ACTIVAR bomba;	Válida	Válida	NOT, OR, grupo y varias acciones.
V06	WHEN humedad < 30 THEN ACTIVAR bomba; WHEN humedad >= 70 THEN DESACTIVAR bomba;	Válida	Válida	Dos reglas consecutivas.
V07	WHEN a == 1 OR b == 2 OR c == 3 THEN ACTIVAR bomba;	Válida	Válida	Cadena de OR.
V08	WHEN a == 1 OR b == 2 AND c == 3 THEN ACTIVAR bomba;	Válida	Válida	Precedencia AND sobre OR.
V09	WHEN NOT NOT lluvia == TRUE THEN DESACTIVAR bomba;	Válida	Válida	Doble NOT.
V10	WHEN (a < 1 OR (b > 2 AND c != 3)) THEN ACTIVAR x Y DESACTIVAR ventilador Y REGAR z POR 5 MIN;	Válida	Válida	Grupos anidados y tres acciones.

10.3 Casos inválidos (mínimo 10)

ID	Entrada	Esperado	Obtenido	Observación
I01	WHEN humedad < 30 ACTIVAR bomba;	Error sintáctico	Error sintáctico (1:19)	Falta THEN.
I02	WHEN humedad < THEN ACTIVAR bomba;	Error sintáctico	Error sintáctico (1:16)	Falta el valor.
I03	WHEN humedad < 30 THEN ACTIVAR bomba	Error sintáctico	Error sintáctico (1:37)	Falta ;.
I04	WHEN (humedad < 30 THEN ACTIVAR bomba;	Error sintáctico	Error sintáctico (1:20)	Falta).
I05	WHEN humedad < 30 THEN REGAR norte 10 MIN;	Error sintáctico	Error sintáctico (1:36)	Falta POR.
I06	WHEN humedad < 30 THEN REGAR norte POR 10;	Error sintáctico	Error sintáctico (1:42)	Falta MIN.
I07	WHEN humedad < limite THEN ACTIVAR bomba;	Error sintáctico	Error sintáctico (1:16)	Valor ID no admitido.
I08	WHEN TRUE THEN ACTIVAR bomba;	Error sintáctico	Error sintáctico (1:6)	Booleano aislado.
I09	WHEN humedad < 30 THEN ACTIVAR;	Error sintáctico	Error sintáctico (1:31)	Falta destino ID.
I10	WHEN humedad < 30 THEN ACTIVAR bomba Y;	Error sintáctico	Error sintáctico (1:39)	Falta acción después de Y.

10.4 Casos límite (mínimo 5)

ID	Entrada	Esperado	Obtenido	Observación
L01	(cadena vacía)	Error sintáctico	Error sintáctico (1:1)	Programa vacío: se exige WHEN.
L02	when humedad < 30 then activar bomba;	Válida	Válida	Reservadas en minúsculas.
L03	WHEN humedad < 30 THEN ACTIVAR bomba; WHEN a @ 3 THEN ACTIVAR b;	Error léxico	Error léxico (2:8)	Símbolo @ en segunda línea.
L04	WHEN a < 1 THEN ACTIVAR b; WHEN a < 1 ACTIVAR b;	Error sintáctico	Error sintáctico (2:12)	Falta THEN en segunda línea.
L05	WHEN a < 1 THEN ACTIVAR b; basura	Error sintáctico	Error sintáctico (1:28)	Texto sobrante después de una regla.

10.5 Análisis de resultados

La ejecución desde una copia limpia de los archivos versionados registró 44 métodos de prueba correctos, sin fallos ni errores: 21 del análisis formal, 14 del lexer/parser/AST y 9 de integración. Los métodos agrupan varias entradas mediante subtests; por eso esa cantidad no debe confundirse con el tamaño del corpus. El corpus de archivos contiene 25 entradas —10 válidas, 10 inválidas y 5 límite— y las 25 coinciden con lo esperado, tanto mediante la API como mediante procesos nuevos del CLI. La tabla calculada no tiene conflictos.

Las pruebas adicionales comprueban conflictos en otras gramáticas, conservación de alternativas, precedencia, asociación, posiciones de error, lectura UTF-8 y secuencias de 1.100 comparaciones con AND/OR. Los resultados y transcripciones se guardan en resultados/; manifest_ejecucion.json registra fecha UTC y huellas SHA-256 de fuentes y entradas, normalizando CRLF a LF. Esta evidencia valida los casos ensayados, pero no demuestra corrección para toda entrada posible. Persisten la ausencia de análisis semántico y el límite de anidación recursiva de paréntesis y NOT.

11. Conclusiones

Se diseñó un lenguaje de automatización agrícola y se implementó un analizador predictivo manual coherente con su gramática. La separación de niveles de expresión permitió fijar la precedencia lógica; las transformaciones explicadas evitan recursión izquierda y prefijos comunes. FIRST y FOLLOW conectan el diseño formal con las decisiones de reconocimiento, y la tabla LL(1) sin conflictos permite elegir una producción con un token de anticipación.

La implementación conserva la agrupación en el AST y entrega errores localizados. Las pruebas del módulo formal, del parser y de integración, junto con el corpus reproducible, aportan evidencia de correspondencia entre especificación y código. Como mejora futura se plantea validar tipos, nombres de sensores y límites de riego, además de ampliar la recuperación de errores. Cualquier conexión a equipos reales requeriría una etapa de evaluación y seguridad fuera del alcance de este proyecto.

Se utilizó IA generativa como apoyo en el diseño, la programación, la documentación y la revisión. Su uso no sustituye la responsabilidad de los integrantes de comprender, justificar y defender las decisiones y resultados presentados.

12. Anexos

Anexo A. Repositorio y recursos

Recurso	Enlace o ubicación
Repositorio GitHub	https://github.com/DiegoSanhueza13/Proyecto_Analisis_Sintactico
Código fuente documentado	https://github.com/DiegoSanhueza13/Proyecto_Analisis_Sintactico/tree/main/src
README	https://github.com/DiegoSanhueza13/Proyecto_Analisis_Sintactico/blob/Nataly/README.md
Archivos auxiliares y corpus de prueba	https://github.com/DiegoSanhueza13/Proyecto_Analisis_Sintactico/tree/main/ejemplos https://github.com/DiegoSanhueza13/Proyecto_Analisis_Sintactico/tree/main/resultados

Anexo B. Demostración funcional

La demostración reproducible utiliza una entrada válida con AST y dos entradas inválidas. Sus salidas reales están en resultados/ejecucion_valida.txt, resultados/error_falta_then.txt y resultados/error_falta_valor.txt. Las figuras B.1 a B.6 muestran capturas de estas ejecuciones y de las pruebas, realizadas en el equipo de un integrante.

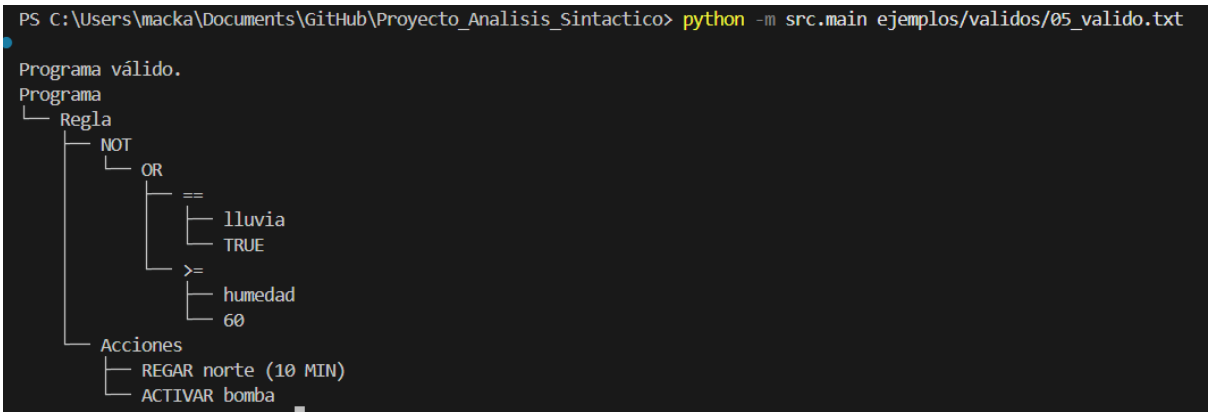


Figura B.1. Entrada válida V05 aceptada por el parser y su AST.

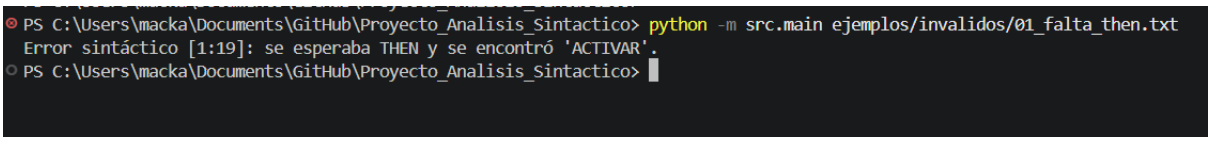


Figura B.2. Entrada inválida I01: error sintáctico por falta de THEN (línea 1, columna 19).

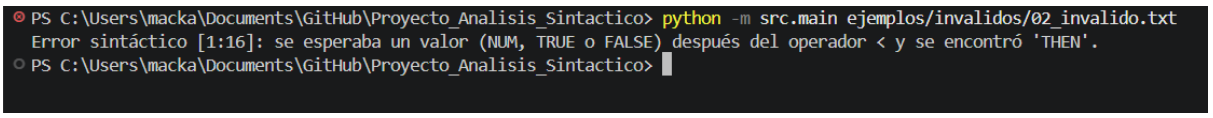


Figura B.3. Entrada inválida I02: error sintáctico por falta de un valor después del operador < (línea 1, columna 16).



Figura B.4. Entrada válida V01: tokens del analizador léxico (posición, tipo y lexema) y AST resultante.

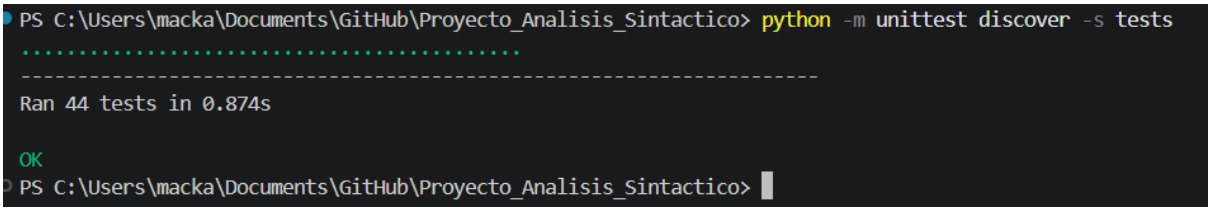


Figura B.5. Ejecución de las 44 pruebas automáticas, todas correctas.

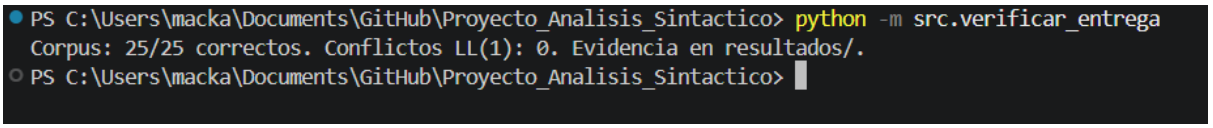


Figura B.6. Verificación del corpus: 25 de 25 entradas correctas y 0 conflictos LL(1).

Anexo C. Reproducción de las pruebas

python -m unittest discover -s tests -v
python -m src.main ejemplos/validos/05_valido.txt --tokens
python -m src.exportar_analisis --salida resultados
python -m src.verificar_entrega

Los comandos se ejecutan desde la raíz del repositorio con Python 3.10 o superior. No se requiere instalar paquetes externos. Para comprobar los errores de la demostración se ejecutan python -m src.main ejemplos/invalidos/01_falta_then.txt y python -m src.main ejemplos/invalidos/02_invalido.txt. La evidencia debe regenerarse si cambian las fuentes o los ejemplos. El informe se basa en los commits c0ce1c5 (módulo formal), c18d9bf (lexer, parser, AST y pruebas) y su integración en main hasta 678afbf.